04. Beacon Chain 상태 관리 및 처리

개요

Beacon Chain은 Lighthouse의 핵심 컴포넌트로, Ethereum 2.0 합의 프로토콜의 상태를 관리하고 블록 및 증명(attestation) 처리를 담당합니다. 이 문서에서는 `BeaconChain` 구조체의 핵심 필드들과 메서드들을 분석하고, 실제 상태 전이 과정을 상세히 설명합니다.

BeaconChain 구조체 분석

핵심 필드 구성

`BeaconChain<T: BeaconChainTypes>` 구조체는 다음과 같은 주요 필드들로 구성됩니다:

```
// beacon_node/beacon_chain/src/beacon_chain.rs
pub struct BeaconChain<T: BeaconChainTypes> {
// 체인 사양 및 설정
pub spec: Arc<ChainSpec>,
pub config: ChainConfig,
// 저장소 및 실행 환경
pub store: BeaconStore<T>,
pub task_executor: TaskExecutor,
pub store_migrator: BackgroundMigrator<T::EthSpec, T::HotStore, T::ColdStore>,
// 시간 관리
pub slot_clock: T::SlotClock,
```

```
// 연산 풀 (Operation Pools)
pub  op_pool: OperationPool<T::EthSpec>,
pub  naive_aggregation_pool: RwLock<NaiveAggregationPool<AggregatedAttestationMa
pub  naive_sync_aggregation_pool: RwLock<NaiveAggregationPool<SyncContributionAg
// 관찰된 객체들 추적
pub(crate) observed_attestations: RwLock<ObservedAggregateAttestations<T::EthSpe
pub  observed_gossip_attesters: RwLock<ObservedAttesters<T::EthSpec>>,
pub  observed_block_attesters: RwLock<ObservedAttesters<T::EthSpec>>,
pub  observed_block_producers: RwLock<ObservedBlockProducers<T::EthSpec>>,
// 실행 레이어 인터페이스
pub  execution_layer: Option<ExecutionLayer<T::EthSpec>>,
// 정규 헤드 및 포크 선택
pub  canonical_head: CanonicalHead<T>,
// 제네시스 정보
pub  genesis_block_root: Hash256,
pub  genesis_state_root: Hash256,
pub  genesis_validators_root: Hash256,
pub  genesis_time: u64,
// 캐시들
pub  shuffling_cache: RwLock<ShufflingCache>,
pub  beacon_proposer_cache: Arc<Mutex<BeaconProposerCache>>,
pub(crate) validator_pubkey_cache: RwLock<ValidatorPubkeyCache<T>>,
pub(crate) attester_cache: Arc<AttesterCache>,
```

```
pub early_attester_cache: EarlyAttesterCache<T::EthSpec>,
// 데이터 가용성 및 KZG
pub data_availability_checker: Arc<DataAvailabilityChecker<T>>,
pub kzg: Arc<Kzg>,
}
```

주요 필드별 역할

1. 상태 저장 및 관리

- **`store`** : 블록, 상태, 기타 데이터의 영구 저장소 (일반적으로 LevelDB)
- **`canonical_head`** : 정규 헤드와 최종화/정당화 체크포인트 정보 저장
- **`genesis_*`** : 제네시스 블록, 상태, 검증자 루트 정보

2. 연산 풀 관리

- **`op_pool`** : 블록에 포함될 후보 연산들 (증명, 예치금, 탈퇴 등) 저장
- **`naive_aggregation_pool`** : 단일 집계 비트를 가진 증명들의 집계 풀
- **`naive_sync_aggregation_pool`** : 동기화 위원회 기여분들의 집계 풀

3. 중복 방지 및 추적

- **`observed_*`** : 이미 처리된 객체들을 추적하여 중복 처리 방지
- **`validator_monitor`** : 특정 검증자들의 성능 모니터링

핵심 메서드 분석

1. process_block() - 블록 처리

`process_block` 메서드는 새로운 블록을 검증하고 체인에 추가하는 핵심 함수입니다:

```
// beacon_node/beacon_chain/src/beacon_chain.rs
pub async fn process_block<B: IntoExecutionPendingBlock<T>>(
self: &Arc<Self>,
block_root: Hash256,
unverified_block: B,
notify_execution_layer: NotifyExecutionLayer,
block_source: BlockImportSource,
publish_fn: impl FnOnce() -> Result<(), BlockError>,
) -> Result<AvailabilityProcessingStatus, BlockError>
```

블록 처리 과정

1. **타이밍 측정 시작**

```
let _full_timer = metrics::start_timer(&metrics::BLOCK_PROCESSING_TIMES);
metrics::inc_counter(&metrics::BLOCK_PROCESSING_REQUESTS);
```

2. **관찰 시간 설정**

```
if let Some(seen_timestamp) = self.slot_clock.now_duration() {
self.block_times_cache.write().set_time_observed(
block_root,
block_slot,
seen_timestamp,
```

```
None,
None,
);
}
```

3. **실행 대기 블록으로 변환**

```
let execution_pending = unverified_block.into_execution_pending_block(
block_root,
&chain,
notify_execution_layer,
)?;
```

4. **합의 검증 완료 시간 기록**

```
if let Some(timestamp) = self.slot_clock.now_duration() {
self.block_times_cache
.write()
.set_time_consensus_verified(block_root, block_slot, timestamp)
}
```

5. **실행된 블록으로 변환**

```
let executed_block = chain.into_executed_block(execution_pending).await?;
```

6. **블록 가용성 확인 및 가져오기**

```
match executed_block {
ExecutedBlock::Available(block) => {
self.import_available_block(Box::new(block)).await
}
ExecutedBlock::AvailabilityPending(block) => {
self.check_block_availability_and_import(block).await
}
}
```

2. 증명 검증 메서드들

verify_unaggregated_attestation_for_gossip()

개별 증명을 gossip 네트워크에서 검증하는 메서드:

```
pub fn verify_unaggregated_attestation_for_gossip<'a>(
&self,
unaggregated_attestation: &'a SingleAttestation,
subnet_id: Option<SubnetId>,
) -> Result<VerifiedUnaggregatedAttestation<'a, T>, AttestationError>
```

검증 과정:

1. 메트릭 카운터 증가
2. 타이머 시작

3. `VerifiedUnaggregatedAttestation::verify()` 호출

4. 이벤트 핸들러에 등록 (SSE 이벤트 발생)

5. 성공 메트릭 업데이트

verify_aggregated_attestation_for_gossip()

집계된 증명을 검증하는 메서드:

```
pub fn verify_aggregated_attestation_for_gossip<'a>(
&self,
signed_aggregate: &'a SignedAggregateAndProof<T::EthSpec>,
) -> Result<VerifiedAggregatedAttestation<'a, T>, AttestationError>
```

배치 검증 메서드들

성능 최적화를 위한 배치 BLS 검증:

```
// 개별 증명들의 배치 검증
pub fn batch_verify_unaggregated_attestations_for_gossip<'a, I>(
&self,
attestations: I,
) -> Result<Vec<Result<VerifiedUnaggregatedAttestation<'a, T>, AttestationError>


// 집계된 증명들의 배치 검증
pub fn batch_verify_aggregated_attestations_for_gossip<'a, I>(
&self,
aggregates: I,
```

```
) -> Result<Vec<Result<VerifiedAggregatedAttestation<'a, T>, AttestationError>>,
```

상태 전이 처리

per_block_processing() 함수

상태 전이의 핵심인 `per_block_processing` 함수는 `consensus/state_processing/src/per_block_processing.rs` 에 구현되어 있습니다:

```
pub fn per_block_processing<E: EthSpec, Payload: AbstractExecPayload<E>>(
state: &mut BeaconState<E>,
signed_block: &SignedBeaconBlock<E, Payload>,
block_signature_strategy: BlockSignatureStrategy,
verify_block_root: VerifyBlockRoot,
ctxt: &mut ConsensusContext<E>,
spec: &ChainSpec,
) -> Result<(), BlockProcessingError>
```

상태 전이 단계

1. **포크 일관성 검증**

```
signed_block
.fork_name(spec)
.map_err(BlockProcessingError::InconsistentBlockFork)?;
```

```
state
.fork_name(spec)
.map_err(BlockProcessingError::InconsistentStateFork)?;
```

2. **에포크 캐시 초기화**

```
initialize_epoch_cache(state, spec)?;
initialize_progressive_balances_cache(state, spec)?;
state.build_slashings_cache()?;
```

3. **서명 검증 전략 결정**

```
let verify_signatures = match block_signature_strategy {
BlockSignatureStrategy::VerifyBulk => {
// 블록의 모든 서명을 한 번에 검증
block_verify!(
BlockSignatureVerifier::verify_entire_block(
state, |i| get_pubkey_from_state(state, i),
|pk_bytes| pk_bytes.decompress().ok().map(Cow::Owned),
signed_block, ctxt, spec
).is_ok(),
BlockProcessingError::BulkSignatureVerificationFailed
);
VerifySignatures::False
}
```

```
BlockSignatureStrategy::VerifyIndividual => VerifySignatures::True,
BlockSignatureStrategy::NoVerification => VerifySignatures::False,
BlockSignatureStrategy::VerifyRandao => VerifySignatures::False,
};
```

4. **블록 헤더 처리**

```
let proposer_index = process_block_header(
state,
block.temporary_block_header(),
verify_block_root,
ctxt,
spec,
)?;
```

5. **위원회 캐시 구축**

```
state.build_committee_cache(RelativeEpoch::Previous, spec)?;
state.build_committee_cache(RelativeEpoch::Current, spec)?;
```

6. **실행 페이로드 처리** (실행 활성화된 경우)

```
if is_execution_enabled(state, block.body()) {
let body = block.body();
process_withdrawals::<E, Payload>(state, body.execution_payload()?, spec)?;
```

```
process_execution_payload::<E, Payload>(state, body, spec)?;
}
```

7. **순차적 처리**

```
process_randao(state, block, verify_randao, ctxt, spec)?;
process_eth1_data(state, block.body().eth1_data())?;
process_operations(state, block.body(), verify_signatures, ctxt, spec)?;
```

8. **동기화 집계 처리** (Altair 이후)

```
if  let Ok(sync_aggregate) = block.body().sync_aggregate() {
process_sync_aggregate(
state,
sync_aggregate,
proposer_index,
verify_signatures,
spec,
)?;
}
```

상태 관리 메서드들

상태 조회 메서드들

```
// 특정 상태 루트로 상태 가져오기
pub fn get_state(&self, state_root: &Hash256, slot: Option<Slot>) -> Result<Op

// 특정 슬롯의 상태 가져오기
pub fn state_at_slot(&self, slot: Slot, config: StateSkipConfig) -> Result<Opt

// 현재 벽시계 시간의 상태 가져오기
pub fn wall_clock_state(&self) -> Result<BeaconState<T::EthSpec>, Error>

// 동기화 위원회 기간의 상태 가져오기
pub fn state_for_sync_committee_period(&self, sync_committee_period: u64) -> R
```

상태 루트 관리

```
// 특정 슬롯의 상태 루트 가져오기
pub fn state_root_at_slot(&self, request_slot: Slot) -> Result<Option<Hash256>

// 상태 루트 반복자들
pub fn rev_iter_state_roots_from<'a>(&'a self, state_root: Hash256) -> StateRo

pub fn forwards_iter_state_roots(&self, start_slot: Slot) -> Result<impl Itera
```

포크 선택 (Fork Choice) 통합

BeaconChain은 `canonical_head` 필드를 통해 포크 선택 알고리즘과 긴밀하게 통합됩니다:

```
pub canonical_head: CanonicalHead<T>,
```

포크 선택 업데이트 과정

1. **새 블록 추가 시**

- 포크 선택 저장소에 블록 추가
- 증명 처리 및 가중치 업데이트
- 새로운 헤드 계산

2. **증명 처리 시**

- 증명의 유효성 검증
- 포크 선택 가중치에 반영
- 필요시 헤드 재계산

3. **정당화/최종화 업데이트**

- 체크포인트 상태 변경 감지
- 포크 선택 저장소 업데이트
- 최종화된 블록들의 정리

성능 최적화 기법

1. 캐싱 전략

```rust
// 셔플링 캐시 - 에포크별 검증자 셔플링 결과 캐시
pub shuffling_cache: RwLock<ShufflingCache>,

// 제안자 캐시 - 블록 제안자 정보 캐시
pub beacon_proposer_cache: Arc<Mutex<BeaconProposerCache>>,

// 검증자 공개키 캐시 - 인덱스 -> 공개키 매핑 캐시
pub(crate) validator_pubkey_cache: RwLock<ValidatorPubkeyCache<T>>,

// 증명자 캐시 - 증명 생성 시 사용되는 캐시
pub(crate) attester_cache: Arc<AttesterCache>,
```

2. 조기 증명자 캐시

```rust
// 헤드 블록이 아직 가져오는 중일 때 증명 생성을 위한 캐시
pub early_attester_cache: EarlyAttesterCache<T::EthSpec>,
```

3. 배치 처리

- **배치 BLS 검증**: 여러 서명을 한 번에 검증하여 CPU 시간 절약
- **배치 상태 전이**: 여러 블록을 연속으로 처리할 때 최적화

에러 처리 및 복구

주요 에러 타입들

```
pub enum BeaconChainError {
InsufficientValidators,
UnableToReadSlot,
InvalidStateForOperation(InvalidStateForOperation),
MissingBeaconState(Hash256),
// ... 기타 에러들
}
```

복구 메커니즘

1. **체크포인트 동기화**: 신뢰할 수 있는 체크포인트에서 시작
2. **백워드 동기화**: 누락된 블록들을 역순으로 다운로드
3. **상태 재구성**: 블록들로부터 상태 재계산

결론

BeaconChain은 Lighthouse의 핵심 컴포넌트로서 복잡한 상태 관리와 블록 처리 로직을 담당합니다. 주요 특징은:

1. **모듈화된 설계**: 각 기능이 명확히 분리되어 유지보수성 향상
2. **성능 최적화**: 다양한 캐싱 전략과 배치 처리로 성능 최적화
3. **견고한 에러 처리**: 다양한 실패 상황에 대한 복구 메커니즘 제공
4. **확장 가능성**: 새로운 포크와 기능 추가에 유연하게 대응

이러한 설계를 통해 BeaconChain은 Ethereum 2.0 합의 프로토콜의 복잡한 요구사항을 효율적으로 처리할 수 있습니다.